

うにマッチ！ アプリ開発・運用マニュアル

～ AI (Antigravity) 連携 & 外部エンジニア共同開発の注意点 ～

1. AI (Antigravity) がコードを変更した際の手順

Antigravityがプログラムを書き換えた後、その変更を本番環境（Vercel）やご自身のパソコンに適用する手順です。

① コミット（保存）の完了を待つ

Antigravityが「コードの書き換えとローカルでのコミットが完了しました」と報告するのをお待ちください。変更内容はまずAI側の作業環境に保存されます。

② 本番環境へのアップロード（プッシュ）

ご自身のパソコンで PowerShell を開き、以下のコマンドを実行します。

```
cd C:\Users%fumij¥.gemini¥antigravity¥scratch¥uni-match-app
git push origin main
```

これで GitHub にコードが送信され、Vercel での自動ビルド（デプロイ）が開始されます。

③ ブラウザでの確認とキャッシュクリア（重要）

「git push」が成功しても、画面に反映されない場合はブラウザが古いデータを記憶（キャッシュ）している可能性が高いです。以下のキーを押して強制再読み込みしてください。

- ・ Windows: Ctrl + F5（または Ctrl + Shift + R）
- ・ Mac: Cmd + Shift + R

④ ローカル開発環境（localhost:3000）で確認している場合

もし「localhost:3000」で動作確認している場合は、PowerShell画面で「Ctrl + C」を押して一度サーバーを停止させ、再度「npm run dev」を入力して再起動してください。

2. ご友人（外部エンジニア）がコードを書き換える際の厳守事項

ご友人にセキュリティ対策や機能追加を依頼する場合、AIと共同でスムーズに開発を進めるために、以下のルールを必ず伝えて守ってもらってください。

① 直接 main ブランチにプッシュさせないこと

【理由】友達が直接 main ブランチを書き換えると、AI側の変更と衝突（コンフリクト）が発生しやすくなります。

【対策】友達には専用の作業ブランチ（例：security-fix など）を作成してもらい、GitHub上で「Pull Request（プルリクエスト）」を作成して main ブランチにマージするフローを徹底してもらってください。

② 友達の変更を必ず Antigravity (AI) へ同期すること（最重要）

【理由】友達がコードを変更した後、それをAIに同期させないまま指示を出すと、AIは古いコードのまま作業を続けてしまい、友達がせっかく修正した箇所を上書きして消してしまいます。

【対策】友達の書き換え作業が完了しGitHub（mainブランチ）に反映されたら、AIに新しい指示を出す前に、ご自身のPowerShellで以下のコマンドを実行してください。

```
git pull origin main
```

これにより、パソコン（AIの作業環境）が最新状態になります。その後、AIに「最新のコードを git pull したよ。友達が〇〇を修正したよ」と教えてください。

③ データベース接続情報やAPIキーの管理

【理由】データベースのパスワードなどの重要な機密情報を直接プログラムコード内に書き込むと、GitHub に公開され漏洩する危険があります。

【対策】接続文字列などの機密情報は、絶対にプログラムファイル内に直接書かず、必ず「.env」ファイルや Vercel の「Environment Variables」設定画面で管理してください。

④ パッケージ追加やスキーマ変更があった場合

- ・友達が新しいライブラリを追加した（package.json が変更された）場合、ローカルで「npm install」を実行してください。

- ・データベースの構造（prisma/schema.prisma）を変更した場合は、「npx prisma db push」や「npx prisma generate」を実行してデータベースを更新してください。